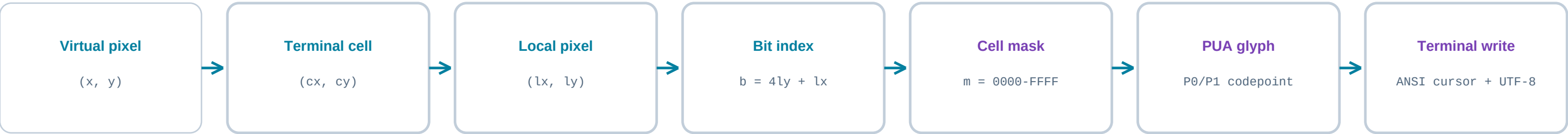


PUA 4x4 Mapping Specification

Expanded educational specification - complete glyph catalog attached

The PUA 4x4 font family converts every cursor-addressable terminal character cell into a programmable 4 x 4 monochrome bitmap. This document keeps four coordinate concepts distinct: a terminal cell, the ANSI cursor address used to reach that cell, a virtual pixel in the enlarged graphical canvas, and a local pixel inside one 4 x 4 cell. The application changes a 16-bit cell mask, maps that mask to a PUA codepoint, and writes the complete replacement glyph at the terminal cursor address.



What this specification teaches

- How terminal columns and rows become a virtual pixel coordinate system four times larger in each axis.
- How one virtual pixel selects one bit in one 16-bit cell mask.
- How mask values select Part 0 or Part 1 and map reversibly to Unicode codepoints.
- How OR, AND NOT and XOR implement set, clear and toggle operations.
- How a shadow framebuffer and dirty-cell updates turn bit operations into terminal output.
- Which terminal, color, aspect-ratio and font-fallback constraints remain.

Terminology: from Unicode text to a visible terminal glyph

These terms describe different layers. Treating codepoint, glyph, character and terminal cell as synonyms makes the mapping difficult to reason about.

Unicode

A universal coded-character standard. It defines codepoints and properties; it does not prescribe one visual drawing for every font.

Codepoint

An integer written as U+ followed by hexadecimal digits. Examples: U+0041 and the project-private U+101669.

Character

An abstract textual element, such as LATIN CAPITAL LETTER A. In casual terminal discussion, 'character' may also mean one cell; this document avoids that ambiguity.

Text

An ordered sequence of encoded characters/codepoints. Example: A followed by B is the sequence U+0041 U+0042.

Font

A resource containing glyph drawings and a cmap that associates supported codepoints with glyph identifiers. Different fonts can draw U+0041 differently.

Glyph

The visible shape selected from a font. PUA 4x4 glyph U+101669 draws mask 0x9669: four corners plus the central 2 x 2 block.

Terminal cell

One fixed grid position addressed by the terminal cursor. A cell stores or displays one glyph cluster plus attributes such as foreground and background color.

UTF-8

A byte encoding for Unicode codepoints. U+0041 is byte 41; U+101669 is the four-byte sequence F4 81 99 A9.

How the layers connect: two concrete examples

normal text: key A -> application outputs U+0041 -> UTF-8 41 -> text font cmap -> glyph A -> terminal cell

PUA graphics: mask 9669 -> U+101669 -> UTF-8 F4 81 99 A9 -> Part 1 cmap -> 4x4 glyph -> terminal cell

Start with the terminal cell, then zoom into its 4 x 4 grid

A blinking text cursor identifies one whole character cell. PUA 4x4 replaces the character in that cell with one 16-bit bitmap glyph.

Terminal cell (cx, cy)

One character position, zero-based in the application.

ANSI cursor (row, column)

The same cell addressed one-based, with row written first.

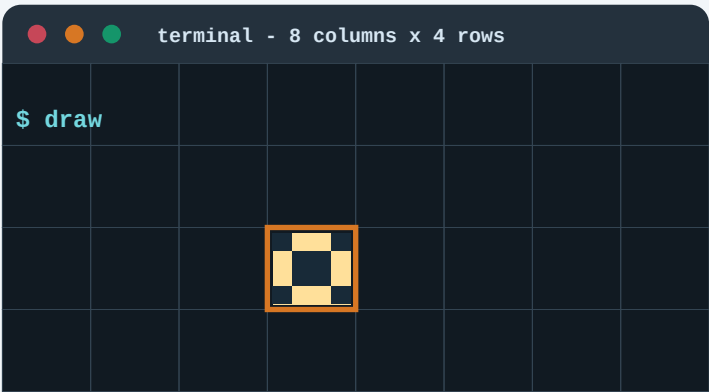
Virtual pixel (x, y)

One point in the 4C x 4R graphical canvas.

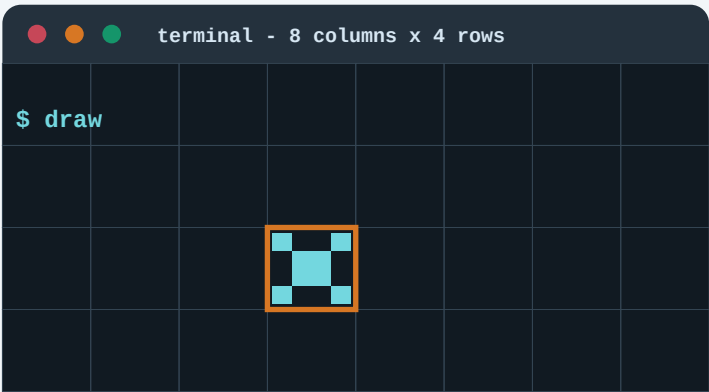
Local pixel (lx, ly)

One of 16 positions inside the selected cell.

A text cursor blinks, but its terminal cell does not move



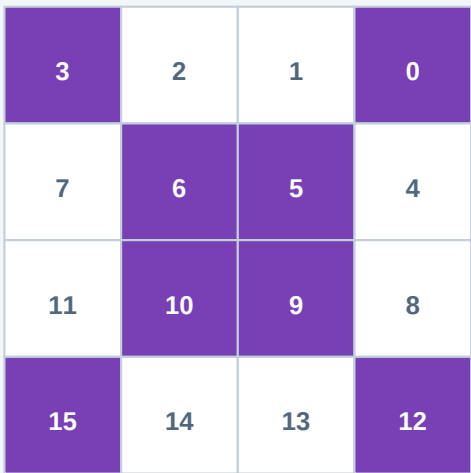
time t0: cursor visible over U+101669



time t1: cursor hidden; glyph remains

Both frames show U+101669 in zero-based application cell (cx, cy) = (3, 2): ANSI row 3, column 4.

Zoom into that one character cell



The glyph is a 16-bit mask.

Columns inside the cell: lx = 0..3

Rows inside the cell: ly = 0..3

Bit index: b = 4 * ly + (3 - lx)

Each number printed in a square is its bit index b.

Filled squares set bits 0,3,5,6,9,10,12,15.

mask = sum(1 << b) for every filled square

= 2^0+2^3+2^5+2^6+2^9+2^10+2^12+2^15

bits 15..0 = 1001 0110 0110 1001 = hex 9 6 6 9

actual mask = 0x9669

That one mask selects one complete glyph: U+101669

The full terminal becomes a larger virtual-pixel canvas

terminal: C columns x R rows

virtual: (4 x C) pixels x (4 x R) pixels

capacity: 16 x C x R independently addressable positions

80 x 24 terminal cells

becomes

320 x 96 virtual pixels

The logical pixels are normally rectangular because terminal character cells are taller than they are wide.

Unicode Private Use Areas and the 65,536-pattern boundary

Private-use codepoints are valid Unicode scalar values whose meaning is defined by a private agreement - here, the published PUA 4x4 mapping convention.

The three Unicode Private Use Areas

Area	Inclusive range	Usable codepoints	How PUA 4x4 uses it
BMP Private Use Area	U+E000-U+F8FF	6,400	unused
Supplementary PUA-A	U+F0000-U+FFFFD	65,534	P0 uses U+F0000-U+F7FFF
Supplementary PUA-B	U+100000-U+10FFFFD	65,534	P1 uses U+100000-U+107FFF
Total private-use capacity	three disjoint ranges	137,468	65,536 assigned by this project

Why 4 x 4 reaches a technical boundary

A 4 x 4 cell has 16 independently selectable positions, so it has 2^16 = 65,536 distinct masks. One supplementary PUA contains 65,534 assignable codepoints - two fewer than required. OpenType glyph indices are also 16-bit; a font must reserve at least glyph 0 for .notdef, so one ordinary font cannot safely provide all 65,536 data glyphs. The design therefore uses two codepoint ranges and two font faces, each with exactly 32,768 project glyphs.

2^16 = 65,536 masks

The bit-15 split is mathematical, not arbitrary

P0 when bit 15 = 0: 0000-7FFF

cp = 0xF0000 + mask

P1 when bit 15 = 1: 8000-FFFF

cp = 0x100000 + (mask - 0x8000)

Because each row is MSB-left, bit 15 is the bottom-left local pixel. Turning that pixel on crosses from P0 to P1 while every other local bit keeps the same numerical meaning.

Boundary examples prove continuity across P0 and P1

Mask	Codepoint	Font	Meaning	Round trip
7FFE	U+0F7FFE	P0	high bit clear; bit 15 off	cp -> mask 7FFE
7FFF	U+0F7FFF	P0	last P0 mask	cp -> mask 7FFF
8000	U+100000	P1	first P1 mask; only bit 15 on	cp -> mask 8000
8001	U+100001	P1	bits 15 and 0 on	cp -> mask 8001
9669	U+101669	P1	four corners plus central 2 x 2	cp -> mask 9669
FFFF	U+107FFF	P1	all 16 local pixels on	cp -> mask FFFF

From a 16-bit mask to Part 0 or Part 1

All bit manipulation happens on the mask first. The final mask value then selects the font part and Unicode codepoint.

Part 0 - high bit clear

$0000 \leq \text{mask} \leq 7FFF$

$\text{codepoint} = 0xF0000 + \text{mask}$

$\text{range} = U+0F0000-U+0F7FFF$

Part 1 - high bit set

$8000 \leq \text{mask} \leq FFFF$

$\text{codepoint} = 0x100000 + (\text{mask} - 0x8000)$

$\text{range} = U+100000-U+107FFF$

Why the split occurs at bit 15

3	2	1	0
7	6	5	4
11	10	9	8
15	14	13	12

Bit 15 is the bottom-left position in MSB-left row order.

A 16-bit glyph mask has 65,536 possible values, but one supplementary Private Use Area segment used here provides only 32,768 consecutive codepoints. Part 0 stores masks whose highest bit is clear. Part 1 stores masks whose highest bit is set. Therefore a glyph is in Part 1 exactly when the bottom-left logical pixel is on. The other 15 pixels may have any combination in either case.

mask 0000 -> U+0F0000 -> Part 0

blank cell

mask 0400 -> U+0F0400 -> Part 0

only bit 10

mask 7FFF -> U+0F7FFF -> Part 0

all except bit 15

Reverse mapping is equally deterministic

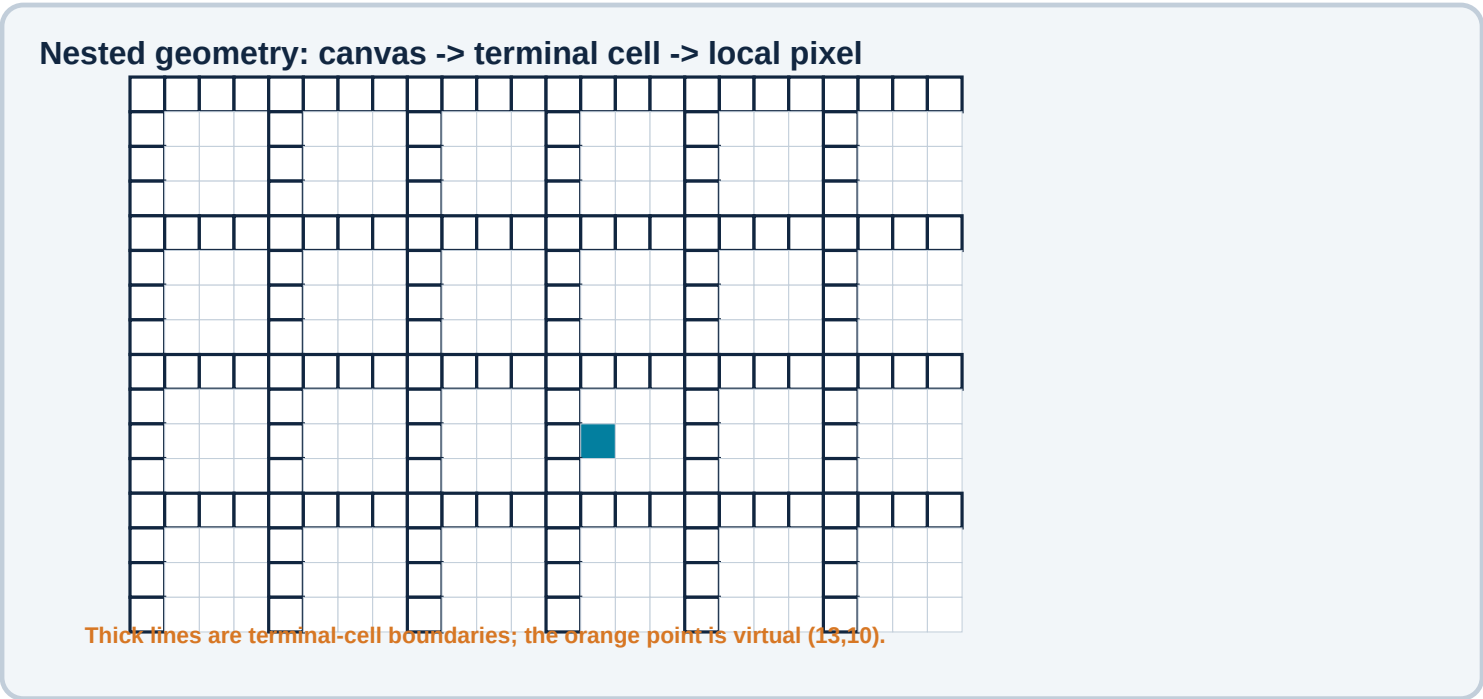
Part 0: $\text{mask} = \text{codepoint} - 0xF0000$

Part 1: $\text{mask} = 0x8000 + (\text{codepoint} - 0x100000)$

Invariant: $\text{mask_to_codepoint}(\text{codepoint_to_mask}(\text{cp})) == \text{cp}$

How all four coordinate systems interact

The same point is described at four layers. Keep column-before-row application tuples separate from row-before-column ANSI syntax.



The equations, including a non-default graphics origin

$cx, lx = \text{divmod}(x, 4)$
 $cy, ly = \text{divmod}(y, 4)$
 $bit = 4 * ly + (3 - lx)$

$\rightarrow \text{divmod}(13, 4) = (3, 1)$
 $\rightarrow \text{divmod}(10, 4) = (2, 2)$
 $\rightarrow 4 * 2 + (3 - 1) = 10$

$ANSI\ row = origin_row + cy$
 $ANSI\ column = origin_column + cx$

$origin\ (row\ 1, col\ 1) \rightarrow ESC[3;4H$
 $origin\ (row\ 5, col\ 10) \rightarrow ESC[7;13H$

Zero-based versus one-based - the practical rule

Graphics algorithms naturally use zero-based arrays: virtual pixel (0,0) is the top-left pixel, terminal cell (0,0) is the top-left cell, and local pixel (0,0) is the top-left subcell. ANSI CUP is a terminal protocol command and normally starts at 1,1. Therefore add the one-based graphics-region origin only when emitting output. Also reverse the tuple order: application data is written (column,row), but ESC[row;columnH writes the row first.

$(cx, cy) = (3, 2)$ is emitted as $ESC[3;4H$ when the region begins at ANSI (row 1, column 1)

Boolean logic for setting, clearing, toggling and combining pixels

A glyph mask is just a 16-bit integer. Each Boolean operation is applied independently to corresponding bit positions.

Single-bit truth table

old A	selector B	A OR B	A AND B	A XOR B	NOT B
0	0	0	0	0	1
0	1	1	0	1	0
1	0	1	0	1	1
1	1	1	1	0	0

OR is 1 when either input is 1. AND is 1 only when both inputs are 1. XOR is 1 when the inputs differ. NOT reverses a bit.

How one selector changes mask 0x9269

SET with OR

```
1001 0010 0110 1001
OR 0000 0100 0000 0000 (0x0400, bit 10)
1001 0110 0110 1001 = 0x9669
```

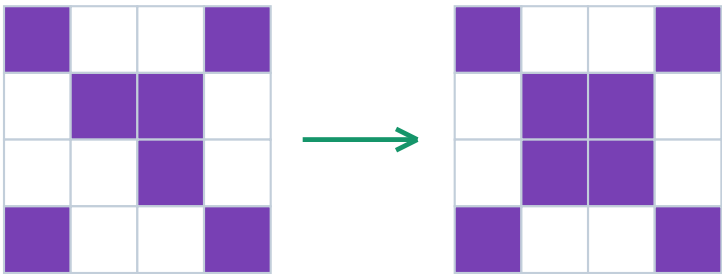
CLEAR with AND NOT

```
1001 0110 0110 1001
AND 1111 1011 1111 1111
1001 0010 0110 1001 = 0x9269
```

TOGGLE with XOR

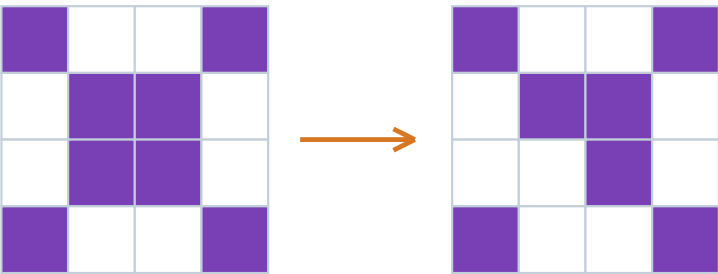
```
1001 0010 0110 1001
XOR 0000 0100 0000 0000 (0x0400, bit 10)
1001 0110 0110 1001 = 0x9669
```

OR: set without disturbing



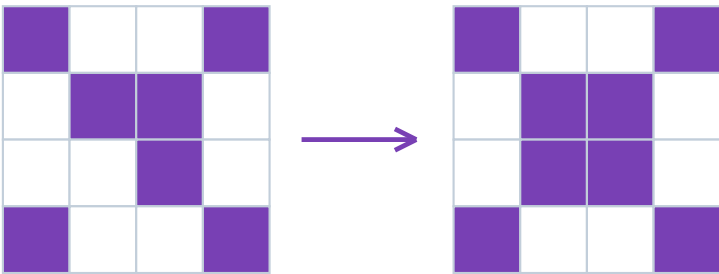
mask |= 0x0400
9269 -> 9669

AND NOT: force clear



mask &= ~0x0400 & 0xFFFF
9669 -> 9269

XOR: invert selected bit



mask ^= 0x0400
9269 -> 9669

Choosing the correct operation

OR Draw/add pixels or combine independent layers. Existing 1 bits remain 1.

AND NOT Erase selected pixels. The final & 0xFFFF keeps Python's unlimited-width complement inside 16 bits.

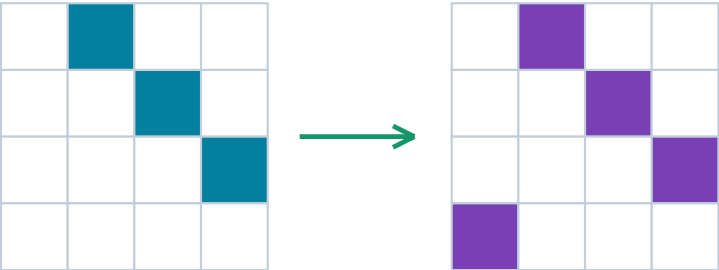
XOR Toggle cursors, selection outlines or reversible overlays. Applying the same XOR twice restores the original mask.

AND Clip a layer to an allowed mask or retain only the pixels shared by two masks.

Bit-level manipulation selects the replacement glyph

The application modifies a cell mask using normal integer logic, then remaps the complete result to one PUA character.

Set pixel: OR



mask |= (1 << 15)

0124 -> 8124

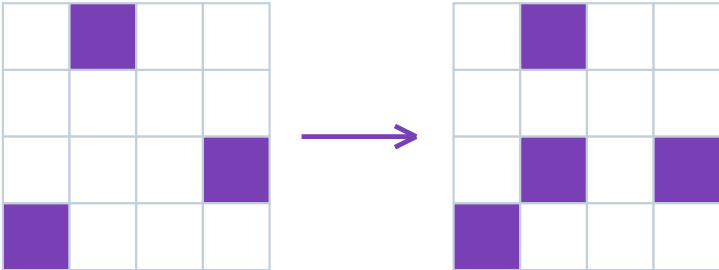
Clear pixel: AND NOT



mask &= ~(1 << 5)

8124 -> 8104

Toggle pixel: XOR



mask ^= (1 << 10)

8104 -> 8504

Worked sequence and resulting codepoints

Operation	Mask calculation	Replacement glyph	Font
Initial	mask = 0x0124	U+0F0124	Part 0
Set bit 15	0x0124 OR 0x8000 = 0x8124	U+100124	Part 1
Clear bit 5	0x8124 AND NOT 0x0020 = 0x8104	U+100104	Part 1
Toggle bit 10	0x8104 XOR 0x0400 = 0x8504	U+100504	Part 1

Other useful operations

Test:	if mask & (1 << bit): pixel is on	Erase:	mask = 0x0000
Replace:	mask = new_mask & 0xFFFF	Fill:	mask = 0xFFFF
Combine:	mask = layer_a layer_b		

PUA 4x4 mapping specification - version 0.8 (MSB-left)

Page 9 of 17

Shadow framebuffer, cursor movement and terminal output

A terminal does not provide a dependable read-back API for displayed glyph state. The application owns the masks and emits changed cells.

Application-owned framebuffer

Store one unsigned 16-bit mask for every terminal character cell. For an 80 x 24 terminal this is an 80-column by 24-row array containing 1,920 masks, or only 3,840 bytes when stored densely. The array is the authoritative graphical state. The terminal is an output device, not the source of truth.

```
uint16 masks[24][80]
```

Dirty-cell rendering

When a virtual pixel operation changes a mask, mark that character cell dirty. At the end of the frame, compare front and back buffers or visit the dirty set. Move the ANSI cursor only to changed cells and write each replacement codepoint as UTF-8. Unchanged cells require no output.

Reference pseudocode

```
def plot(x, y, operation='set'):
    cell_col, local_col = divmod(x, 4)
    cell_row, local_row = divmod(y, 4)
    bit = local_row * 4 + (3 - local_col)
    old = masks[cell_row][cell_col]
    new = old | (1 << bit)           # set; use AND NOT or XOR as required
    if new != old:
        masks[cell_row][cell_col] = new
        dirty.add((cell_col, cell_row))

for cell_col, cell_row in dirty:
    cp = mask_to_codepoint(masks[cell_row][cell_col])
    write(f'\x1b[{origin_row+cell_row};{origin_col+cell_col}H' + chr(cp))
```

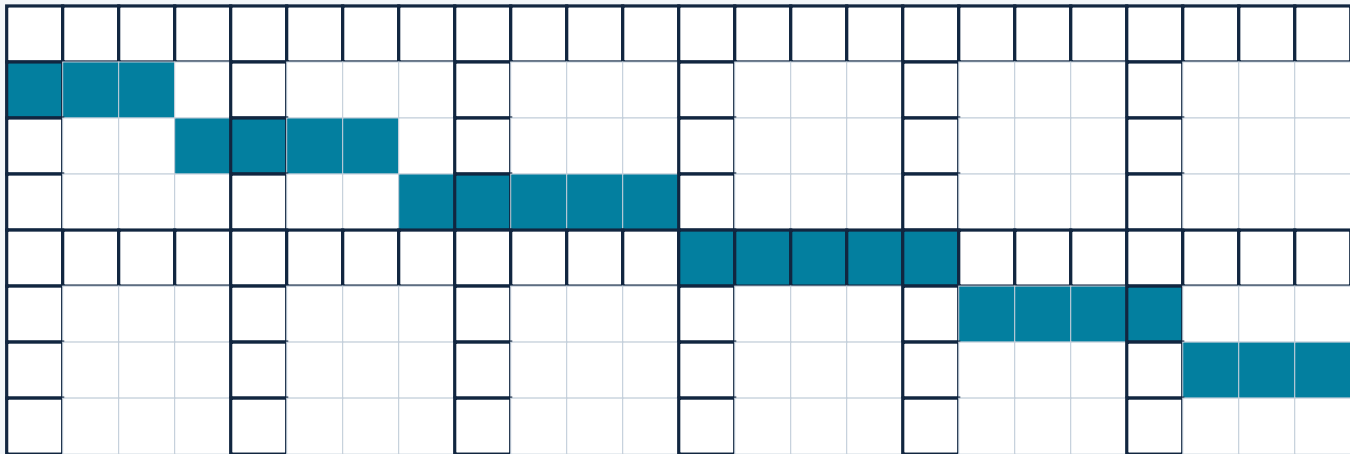
Replacement is cell-granular even though addressing is subcell-granular

Changing one virtual pixel does not patch part of an existing glyph in the terminal. It changes one bit in the application's mask and causes one complete PUA glyph to replace the previous character at that cursor position. All 16 visible logical pixels in that cell are therefore reconstructed from the new mask every time the cell is emitted.

Building programmatic visualizations

Drawing algorithms operate in the 4C x 4R virtual coordinate space and call the same plot operation for every covered position.

A diagonal crossing six character cells



24 x 8 virtual positions encoded by only 6 x 2 terminal characters

Primitive-to-pixel pipeline

Line	Bresenham or DDA -> plot each point
Circle	midpoint circle -> symmetric points
Polygon	edge rasterization + scanline fill
Sprite	OR to add, AND NOT to erase, XOR to toggle
3D wireframe	project vertices -> line segments
Chart	map samples -> virtual x/y coordinates

Why this is more flexible than ordinary terminal characters

- Coordinates are mathematical and independent of language or text semantics.
- Cell masks can be composited, diffed, animated and regenerated deterministically.
- The same rendering core can support charts, vector art, games, particles and telemetry.
- Every 4 x 4 combination exists, so adjacent pixels combine without approximation.
- Existing terminal cursor addressing, UTF-8 transport and ANSI color remain usable.

Animation strategy

Maintain a back buffer for the next frame, compare it with the displayed front buffer, emit only changed glyph cells, then swap buffers. A one-virtual-pixel movement may change one cell or two neighboring cells at a character boundary; the same diff process handles both cases without special terminal logic.

Operational requirements, limits and review checklist

The font extends terminal graphics capability but does not change the terminal protocol or remove renderer constraints.

Required for correct operation

Both font parts

The terminal fallback chain must select PUA 4x4 Part 0 and Part 1 for their respective codepoint ranges.

One-column width

Supplementary-PUA codepoints must be treated as wwidth 1 by the locale, terminal and shaping stack.

UTF-8 and cmap 12

The output path must preserve supplementary-plane Unicode and the fonts require a format-12 character map.

Shadow state

The application must retain cell masks; terminal screen read-back is neither portable nor sufficient.

Resize handling

A resize changes C, R, virtual dimensions and buffer allocation. Rebuild or remap the graphical state.

Constraints to design around

Pixel aspect ratio

The nominal 125 x 250 font-unit subcells are rectangular in a conventional terminal cell.

Color granularity

ANSI foreground color normally applies to the whole glyph cell, not independently to its 16 logical pixels.

Update granularity

One bit change replaces one complete character. Dirty-cell output minimizes the resulting terminal traffic.

Private mapping

These codepoints are a published project convention, not characters standardized by the Unicode Consortium.

Renderer rasterization

Fractional device-pixel font sizes may introduce antialiasing variation even when logical geometry is exact.

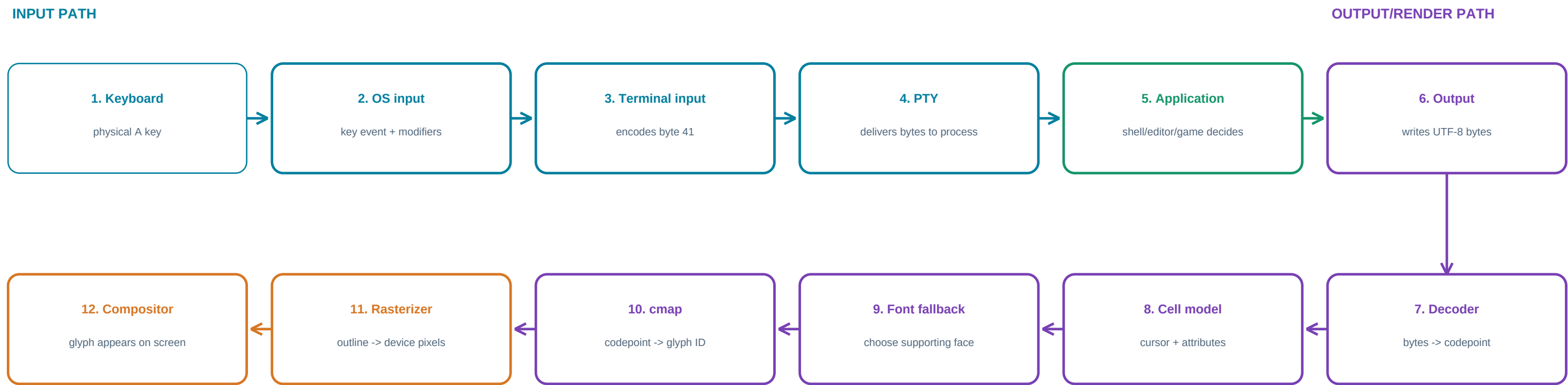
Decisions incorporated in this complete specification

- ☒ Added a terminology section separating Unicode, codepoint, character, text, font, glyph, UTF-8 and terminal cell.
- ☒ Added all three PUA ranges, capacity limits, P0/P1 boundary examples and the significance of bit 15.
- ☒ Added the full executable standard-library reference renderer in Appendix C.
- ☒ Added a dedicated diagram connecting zero-based graphical coordinates with one-based ANSI cursor positions.
- ☒ Added truth tables, binary calculations and practical guidance for OR, AND NOT, XOR and AND.
- ☐ The exhaustive 259-page character specification and 256-page mask catalog are appended unchanged after this section.

End of mapping guide - the exhaustive 65,536-glyph catalog follows

Appendix A - from a keypress to a rendered glyph in a terminal cell

A key does not directly select a font glyph. Input first reaches a program; only the program's output is decoded and rendered by the terminal.



Normal text example: pressing A

keypress A -> input byte 41 -> shell/readline receives it -> echo output byte 41 -> U+0041 -> text font -> glyph A

If echo is disabled, the key can be received without anything appearing. If an application transforms the input, the displayed output may be different from the key. This is why input and rendering are separate paths.

PUA 4x4 example: the application renders mask 0x9669

mask 9669 -> codepoint U+101669 -> UTF-8 F4 81 99 A9 -> terminal decoder -> Part 1 cmap -> 4x4 glyph

No keyboard key is assigned to U+101669. Graphics software computes and outputs it directly as part of the frame.

Appendix B - executable reference renderer

The following three listing pages are a complete standard-library Python program, not pseudocode or an excerpt.

What the renderer demonstrates

- A C x R array of unsigned 16-bit masks acts as the framebuffer.
- divmod maps each virtual (x,y) to a terminal cell and local bit.
- OR, AND NOT and XOR implement set, clear and toggle.
- Bresenham draws lines in the 4C x 4R virtual coordinate space.
- mask_to_codepoint selects P0 or P1 deterministically.
- ANSI CUP positions each row and UTF-8 carries supplementary PUA codepoints.

Run it

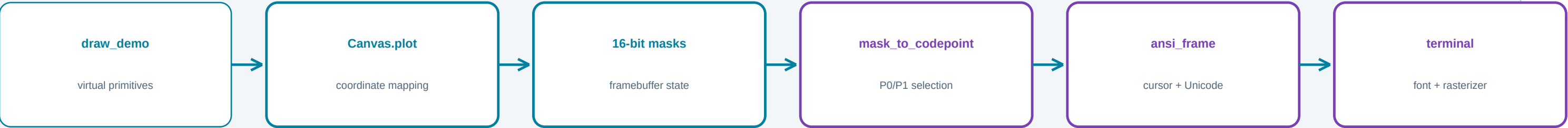
```
# Linux installation used by this project:
cd "$HOME/dev/FontMaker/pua4x4"
test -f pua4x4_reference_renderer.py || exit 1
./launch-linux.sh shell
```

Then, inside the PUA 4x4 terminal:

```
cd "$HOME/dev/FontMaker/pua4x4"
python3 pua4x4_reference_renderer.py \
    --columns 40 --rows 12 --seconds 2 --fps 12
```

The first command block matches the verified Linux deployment path. The Git repository layout may instead use experiments/pua-4x4. In either layout, run the program only in a terminal profile selecting the PUA 4x4 Fontconfig alias. Ctrl-C exits an unbounded run.

Renderer architecture



Scope and production considerations

This reference deliberately redraws every cell so the mapping remains easy to audit. A production renderer should retain front and back buffers, emit only dirty cells, coalesce adjacent updates, handle resize events, measure the terminal pixel aspect ratio and apply frame pacing based on actual output throughput.

Appendix C - complete reference renderer source (1/3)

pua4x4_reference_renderer.py - lines 1-48; reproduce the file by concatenating these listing pages in order.

```
1  #!/usr/bin/env python3
2  """Minimal, executable PUA 4x4 terminal renderer (standard library only)."""
3
4  import argparse
5  import shutil
6  import sys
7  import time
8
9
10 PART0_BASE = 0xF0000
11 PART1_BASE = 0x100000
12 PART_SPLIT = 0x8000
13
14
15 def mask_to_codepoint(mask):
16     """Map one unsigned 16-bit cell mask to the project PUA convention."""
17     if not 0 <= mask <= 0xFFFF:
18         raise ValueError("mask must be between 0x0000 and 0xFFFF")
19     if mask < PART_SPLIT:
20         return PART0_BASE + mask
21     return PART1_BASE + (mask - PART_SPLIT)
22
23
24 class Canvas:
25     """A 4C x 4R virtual-pixel canvas backed by C x R cell masks."""
26
27     def __init__(self, columns, rows):
28         self.columns = columns
29         self.rows = rows
30         self.width = columns * 4
31         self.height = rows * 4
32         self.masks = [[0 for _ in range(columns)] for _ in range(rows)]
33
34     def erase(self):
35         for row in self.masks:
36             row[:] = [0] * self.columns
37
38     def address(self, x, y):
39         """Return cell column/row, bit index and selector for virtual (x,y)."""
40         if not (0 <= x < self.width and 0 <= y < self.height):
41             return None
42         cell_column, local_column = divmod(x, 4)
43         cell_row, local_row = divmod(y, 4)
44         bit = local_row * 4 + (3 - local_column)
45         return cell_column, cell_row, bit, 1 << bit
46
47     def plot(self, x, y, operation="set"):
48         location = self.address(x, y)
```

Appendix C - complete reference renderer source (2/3)

pua4x4_reference_renderer.py - lines 49-96; reproduce the file by concatenating these listing pages in order.

```
49         if location is None:
50             return
51         cell_column, cell_row, _bit, selector = location
52         old = self.masks[cell_row][cell_column]
53         if operation == "set":
54             new = old | selector
55         elif operation == "clear":
56             new = old & ~selector & 0xFFFF
57         elif operation == "toggle":
58             new = old ^ selector
59         else:
60             raise ValueError("operation must be set, clear or toggle")
61         self.masks[cell_row][cell_column] = new
62
63     def line(self, x0, y0, x1, y1, operation="set"):
64         """Integer Bresenham line in virtual-pixel coordinates."""
65         dx = abs(x1 - x0)
66         sx = 1 if x0 < x1 else -1
67         dy = -abs(y1 - y0)
68         sy = 1 if y0 < y1 else -1
69         error = dx + dy
70         while True:
71             self.plot(x0, y0, operation)
72             if x0 == x1 and y0 == y1:
73                 return
74             doubled = 2 * error
75             if doubled >= dy:
76                 error += dy
77                 x0 += sx
78             if doubled <= dx:
79                 error += dx
80                 y0 += sy
81
82     def ansi_frame(self, origin_row=1, origin_column=1):
83         """Encode all masks as cursor-addressed UTF-8 terminal text."""
84         output = []
85         for cell_row, row in enumerate(self.masks):
86             ansi_row = origin_row + cell_row
87             output.append(f"\x1b[{ansi_row};{origin_column}H")
88             output.append("".join(chr(mask_to_codepoint(mask)) for mask in row))
89         return "".join(output)
90
91
92     def draw_demo(canvas, phase):
93         """Draw a border, diagonals and a moving cross using virtual pixels."""
94         canvas.erase()
95         right = canvas.width - 1
96         bottom = canvas.height - 1
```


Appendix C - complete reference renderer source (3/3)

pua4x4_reference_renderer.py - lines 97-144; reproduce the file by concatenating these listing pages in order.

```
97     canvas.line(0, 0, right, 0)
98     canvas.line(right, 0, right, bottom)
99     canvas.line(right, bottom, 0, bottom)
100    canvas.line(0, bottom, 0, 0)
101    canvas.line(0, 0, right, bottom)
102    canvas.line(0, bottom, right, 0)
103    moving_x = int((phase % 1.0) * right)
104    moving_y = int((phase * 0.63) % 1.0) * bottom)
105    for offset in range(-5, 6):
106        canvas.plot(moving_x + offset, moving_y)
107        canvas.plot(moving_x, moving_y + offset)
108
109
110    def main():
111        parser = argparse.ArgumentParser()
112        parser.add_argument("--columns", type=int)
113        parser.add_argument("--rows", type=int)
114        parser.add_argument("--fps", type=float, default=12.0)
115        parser.add_argument("--seconds", type=float, default=0.0,
116                            help="0 runs until Ctrl-C")
117        args = parser.parse_args()
118        terminal = shutil.get_terminal_size((80, 24))
119        columns = args.columns or terminal.columns
120        rows = args.rows or max(1, terminal.lines - 1)
121        canvas = Canvas(columns, rows)
122        started = time.monotonic()
123        sys.stdout.write("\x1b[?1049h\x1b[2J\x1b[?25l")
124        try:
125            while True:
126                frame_started = time.monotonic()
127                elapsed = frame_started - started
128                if args.seconds and elapsed >= args.seconds:
129                    break
130                draw_demo(canvas, elapsed * 0.18)
131                sys.stdout.write(canvas.ansi_frame())
132                sys.stdout.flush()
133                delay = (1.0 / args.fps) - (time.monotonic() - frame_started)
134                if delay > 0:
135                    time.sleep(delay)
136        except KeyboardInterrupt:
137            pass
138        finally:
139            sys.stdout.write("\x1b[?25h\x1b[?1049l")
140            sys.stdout.flush()
141
142
143    if __name__ == "__main__":
144        main()
```